

```

1  #include <stdio.h>
2
3  struct priority_scheduling {
4      char process_name;
5      int burst_time;
6      int waiting_time;
7      int turn_around_time;
8      int priority;
9  };
10
11 int main() {
12     int number_of_process;
13     int total = 0;
14     struct priority_scheduling temp_process;
15     int ASCII_number = 65; // 65 is ASCII for 'A'
16     int position;
17     float average_waiting_time;
18     float average_turnaround_time;
19
20     // 1. Get number of processes
21     printf("Enter the total number of Processes: ");
22     scanf("%d", &number_of_process);
23
24     struct priority_scheduling process[number_of_process];
25
26     // 2. Input details for each process
27     printf("\nPlease Enter the Burst Time and Priority of each process:\n");
28     for (int i = 0; i < number_of_process; i++) {
29         process[i].process_name = (char) ASCII_number;
30         printf("\nEnter the details of process %c:\n", process[i].process_name);
31
32         printf("Enter the burst time: ");
33         scanf("%d", &process[i].burst_time);
34
35         printf("Enter the priority: ");
36         scanf("%d", &process[i].priority);
37
38         ASCII_number++;
39     }
40
41     // 3. Sort processes by priority (Highest Priority First)
42     for (int i = 0; i < number_of_process; i++) {
43         position = i;
44         for (int j = i + 1; j < number_of_process; j++) {
45             if (process[j].priority > process[position].priority) {
46                 position = j;
47             }
48         }
49         temp_process = process[i];
50         process[i] = process[position];
51         process[position] = temp_process;
52     }
53
54     // 4. Calculate Waiting Time (WT)
55     process[0].waiting_time = 0;
56     for (int i = 1; i < number_of_process; i++) {
57         process[i].waiting_time = 0;
58         for (int j = 0; j < i; j++) {
59             process[i].waiting_time += process[j].burst_time;
60         }
61         total += process[i].waiting_time;
62     }
63
64     // Calculate Average Waiting Time
65     average_waiting_time = (float) total / (float) number_of_process;
66
67     // Reset total to calculate Turnaround Time sum
68     total = 0;
69
70     // 5. Print Table Headings
71     printf("\n\nProcess_name \t Burst Time \t Waiting Time \t Turnaround Time\n");
72     printf("-----\n");
73
74     // 6. Calculate Turnaround Time (TAT) and Print Rows
75     for (int i = 0; i < number_of_process; i++) {
76         process[i].turn_around_time = process[i].burst_time + process[i].waiting_time;
77         total += process[i].turn_around_time; // Accumulate total TAT
78
79         printf("\t %c \t\t %d \t\t %d \t\t %d\n",
80             process[i].process_name,
81             process[i].burst_time,
82             process[i].waiting_time,
83             process[i].turn_around_time);
84     }
85     printf("-----\n");
86
87     // 7. Calculate and display averages
88     average_turnaround_time = (float) total / (float) number_of_process;
89
90     printf("\n\nAverage Waiting Time : %f", average_waiting_time);
91     printf("\n\nAverage Turnaround Time: %f\n", average_turnaround_time);
92
93     return 0;
94 }

```

Enter the total number of Processes: 3

Please Enter the Burst Time and Priority of each process:

Enter the details of process A:

Enter the burst time: 2

Enter the priority: 1

Enter the details of process B:

Enter the burst time: 10

Enter the priority: 3

Enter the details of process C:

Enter the burst time: 6

Enter the priority: 2

Process_name	Burst Time	Waiting Time	Turnaround Time
B	10	0	10
C	6	10	16
A	2	16	18

Average Waiting Time : 8.666667

Average Turnaround Time: 14.666667

Process exited after 39.26 seconds with return value 0